Listenify, Type Promotion, Garbage Collection and Comparable

Monday, October 5, 2026 8:09 PM

1. Listenify: album and playlist manager

Listenify is a small Java console application that manages songs, albums and an ordered playlist. Song stores a title and duration. Album stores its name, artist and songs. The playlist contains selected songs from one or more albums. Player navigation selects the current, next or previous song.

The reference application prints playback messages. It does not play audio or connect to a remote server.

Main operations: add a song to an album, reject duplicate titles, add to a playlist by title or track number, show the playlist, move next/previous, replay and remove the current song. Pause/resume can be modeled as player state; printing "paused" is not actual audio playback.

2. Objects and relationships

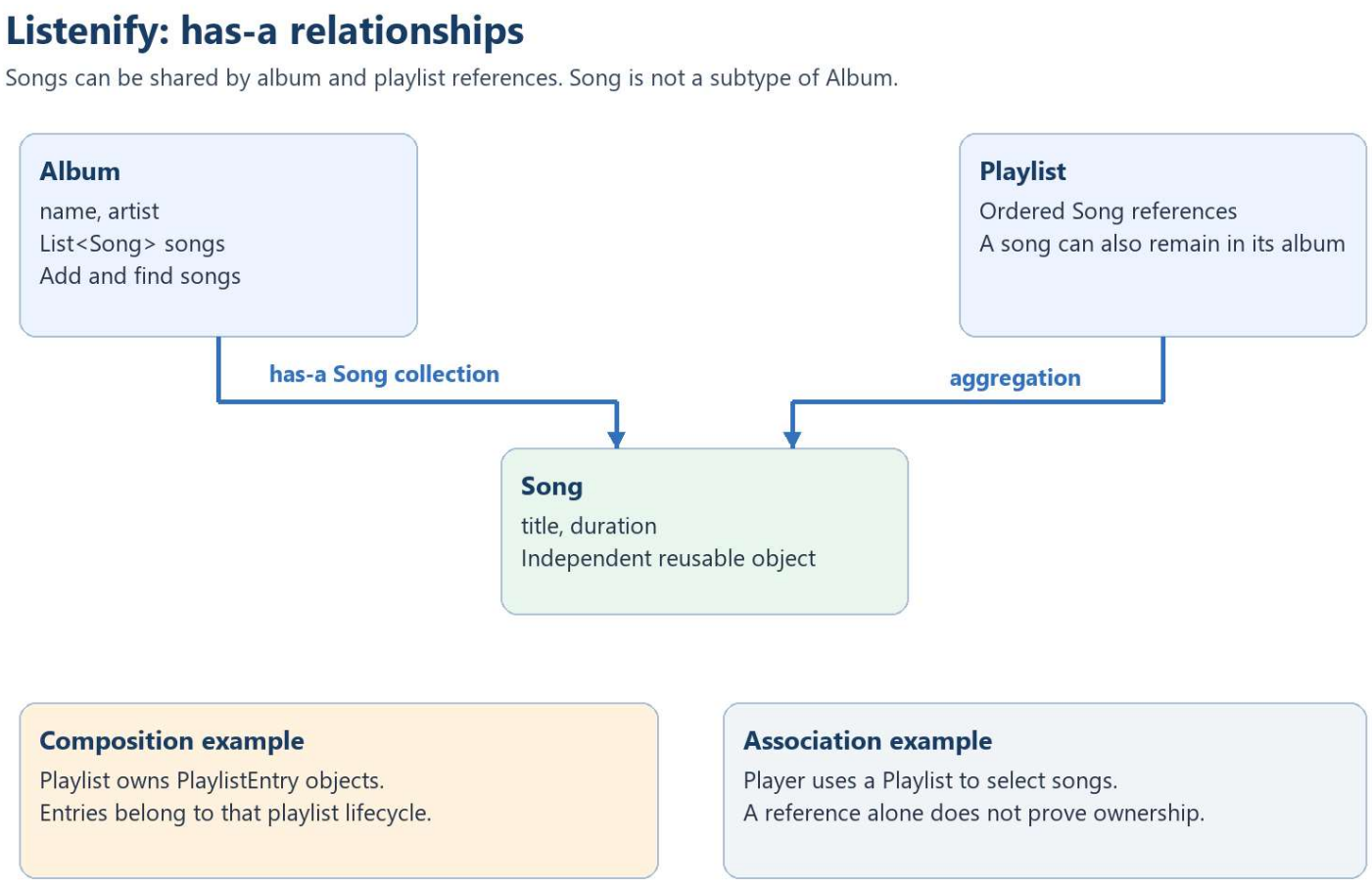
Inheritance represents an "is-a" relationship. A Song is not an Album, so Song should not extend Album.

Association: one object knows about or uses another. A player uses a playlist.

Aggregation: a "has-a" relationship where referenced objects can exist independently. A playlist holds Song references; removing one playlist entry does not delete the song from its album.

Composition: a stronger ownership relationship. A whole owns parts whose application lifecycle belongs to it. A Playlist could own PlaylistEntry objects that are discarded with that playlist, while each entry references an independently existing Song.

In Java, garbage collection follows reachability. Deleting a reference does not immediately destroy an object. A List field by itself does not prove composition; ownership rules determine the relationship.



3. OOP and Java concepts used here

Encapsulation: keep fields private and expose methods that enforce valid operations.

Constructors: initialize each song and album with valid data.

Overloading: addToPlaylist(int trackNo, ...) and addToPlaylist(String title, ...) share a name but accept different argument types.

Overriding: Song overrides Object.toString() to provide a useful printed representation.

Interface types: List<Song> can refer to an ArrayList<Song> or LinkedList<Song> object. A

variable's interface type describes available operations.
Generics: List<Song> restricts elements to Song-compatible references and avoids manual casts when reading them.
The reference app mainly adds object relationships, collections and iterator navigation. It does not introduce a custom inheritance hierarchy like the Bank application.

4. Song example

Duration is measured in decimal minutes here: 3.5 minutes means 3 minutes 30 seconds, not 3 minutes 50 seconds.

```
public final class Song {
    private final String title;
    private final double durationMinutes;

    public Song(String title, double durationMinutes) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title required");
        }
        if (!
Double.isFinite(durationMinutes) || durationMinutes <= 0) {
            throw new IllegalArgumentException("Duration must be posit
ive");
        }
        this.title = title;
        this.durationMinutes = durationMinutes;
    }

    public String getTitle() { return title; }

    @Override
    public String toString() {
        return title + " (" + durationMinutes + " min)";
    }
}
```

5. Album example: a collection of songs

ArrayList preserves insertion order and supports indexed access. Track numbers are 1-based; list indexes are 0-based, so index = trackNo - 1.
The album and playlist may hold references to the same Song object. Adding an existing song to a playlist does not copy the Song.
Use equals() to compare title content. Using == compares references and can fail for equal text stored in different String objects.

```
import java.util.ArrayList;
import java.util.List;

public class Album {
    private final String name;
    private final String artist;
    private final List<Song> songs = new ArrayList<>();

    public Album(String name, String artist) {
        if (name == null || name.isBlank() || artist == null || artist
.isBlank()) {
            throw new IllegalArgumentException("Album name and artist
required");
        }
        this.name = name;
        this.artist = artist;
    }

    private Song findSong(String title) {
        for (Song song : songs) {
            if (song.getTitle().equals(title)) return song;
        }
        return null;
    }

    public void addSong(String title, double minutes) {
        if (findSong(title) != null) {
            throw new IllegalArgumentException("Duplicate song title")
;
        }
        songs.add(new Song(title, minutes));
    }

    public void addToPlaylist(int trackNo, List<Song> playlist) {
        requirePlaylist(playlist);
        int index = trackNo - 1;
        if (index < 0 || index >= songs.size()) {
            throw new IllegalArgumentException("Invalid track number")
;
        }
        playlist.add(songs.get(index));
    }
```

```
    public void addToPlaylist(String title, List<Song> playlist) {
        requirePlaylist(playlist);
        Song song = findSong(title);
        if (song == null) throw new IllegalArgumentException("Song not
 found");
        playlist.add(song);
    }

    private static void requirePlaylist(List<Song> playlist) {
        if (playlist == null) throw new IllegalArgumentException("Play
list required");
    }
}
```

Duplicate titles are rejected within the same album. This example permits the same song to appear more than once in a playlist.

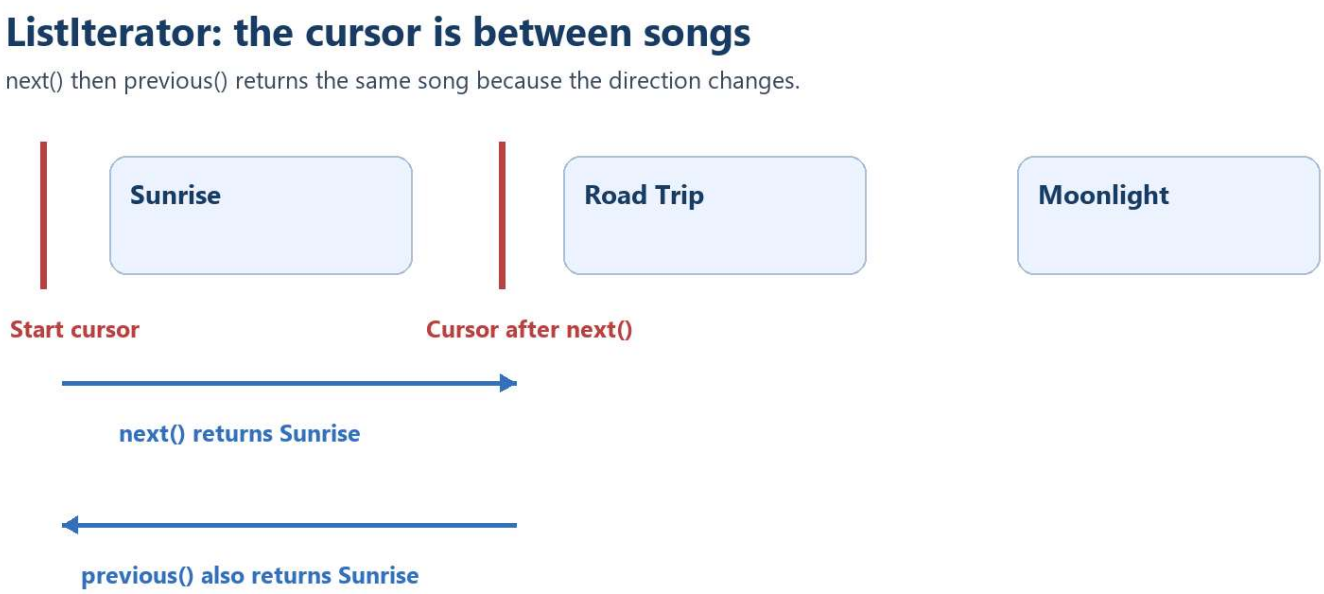
6. ListIterator: a cursor between elements

ListIterator can move in both directions. It has no permanent “current element”; its cursor sits between elements.

next() returns the element after the cursor and moves forward. previous() returns the element before the cursor and moves backward.

Use hasNext() and hasPrevious() before moving. Calling next() and then previous() returns the same element.

remove() removes the element most recently returned by next() or previous(). It requires a traversal since the previous remove/add, otherwise IllegalStateException occurs.



Safe movement and removal

Check hasNext()/hasPrevious() before traversing. remove() removes the last returned element, not an element selected by a separate index. Track player selection separately.

```
import java.util.LinkedList;
import java.util.ListIterator;

public class ListenifyDemo {
    public static void main(String[] args) {
        Album album = new Album("Morning Mix", "Various");
        album.addSong("Sunrise", 3.5);
        album.addSong("Road Trip", 4.0);
        LinkedList<Song> playlist = new LinkedList<>();
        album.addToPlaylist(new String("Sunrise"), playlist);
        album.addToPlaylist(2, playlist);

        ListIterator<Song> cursor = playlist.listIterator();
        if (cursor.hasNext()) System.out.println(cursor.next());
        if (cursor.hasNext()) System.out.println(cursor.next());
        if (cursor.hasPrevious()) System.out.println(cursor.previous()
);
```

```
        cursor.remove(); // Removes Road Trip, returned by previous().
        System.out.println(playlist); // Album still contains Road Tri
p.
    }
}
```

Output: Sunrise (3.5 min), Road Trip (4.0 min), Road Trip (4.0 min), then [Sunrise (3.5 min)].
A full player must track its current selection and traversal direction carefully. Replay, direction changes and removal are different operations. LinkedList is used to explore the reference project's navigation; it is not automatically the best list for every application.

7. Exception handling in Listenify

Validate titles, durations and track numbers before changing collections. Catch NumberFormatException when parsing console input. Reject operations on an empty playlist with a clear response.

```
try {
    int track = Integer.parseInt("abc");
    album.addToPlaylist(track, playlist);
} catch (NumberFormatException ex) {
    System.out.println("Enter a whole-number track");
} catch (IllegalArgumentException ex) {
    System.out.println(ex.getMessage());
}
```

This fragment uses album and playlist from the preceding example. The more specific NumberFormatException catch comes first.
In the reference repository, title-based playlist lookup uses == in one method. Replace it with equals(). Check navigation/removal boundaries and invalid menu input when developing the player.

8. Type promotion and method overloading

An argument can be widened to an accepted primitive type when there is no exact parameter match. Overload selection occurs at compile time.
In the example, an int argument can widen to long or double; long is the more specific applicable overload. A double argument matches double directly.

```
public class TypePromotionDemo {
    static void showDuration(long minutes) {
        System.out.println("long version");
    }
    static void showDuration(double minutes) {
        System.out.println("double version");
    }
    public static void main(String[] args) {
        showDuration(3);   // int -> long
        showDuration(3.5); // double -> double
    }
}
```

Output: long version, then double version.
Java does not automatically narrow a method argument from double to long. Do not choose overloads by memory size: applicability and specificity determine the result. Primitive widening is different from boxing a primitive into a wrapper object.

9. Garbage collection and shared references

Garbage collection is the JVM's automatic reclamation of memory occupied by objects that are no longer reachable. Reachability means there is a live path to an object from the running program's roots.
An album and playlist can reference the same Song. Removing one reference does not destroy that Song while another live reference still reaches it.

Removing a playlist entry does not delete the Song



playlist.clear(): only the playlist references are removed.

The Song remains reachable through the album or any other live reference.

No live path to the Song? It becomes eligible for collection.

Eligible does not mean collected immediately. System.gc() is only a request.

```
import java.util.ArrayList;
import java.util.List;

public class GarbageCollectionDemo {
    public static void main(String[] args) {
        Song song = new Song("Sunrise", 3.5);
        List<Song> albumSongs = new ArrayList<>();
        List<Song> playlist = new ArrayList<>();
        albumSongs.add(song);
        playlist.add(song);

        playlist.clear();  // Album and song variable still refer to i
t.
        song = null;       // Album still refers to it.
        System.out.println(albumSongs); // [Sunrise (3.5 min)]
        albumSongs.clear();// No remaining Song references in this exa
mple.
    }
}
```

The final clear makes the Song eligible for collection; it does not prove that collection happened at that line. System.gc() does not guarantee immediate collection. Do not use finalize() to demonstrate object destruction; it is deprecated for removal. Garbage collection does not replace closing files or network connections.

10. Comparable: define the natural order of Song

Comparable<T> is an interface a class implements to define its natural sorting order. Its compareTo(T other) method returns a negative value, zero or a positive value for before, equal in ordering or after.

For Song, title order is one possible natural order. String.compareTo uses case-sensitive lexicographic ordering; it is not locale-aware dictionary sorting.

Add implements Comparable<Song> to the existing Song declaration, then add this method inside it:

```
@Override
public int compareTo(Song other) {
    return this.title.compareTo(other.title);
}
```

Sort a separate list so the playback order is preserved:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class SongOrderingDemo {
    public static void main(String[] args) {
        List<Song> playlist = new ArrayList<>();
        playlist.add(new Song("Sunrise", 3.5));
        playlist.add(new Song("Moonlight", 4.0));

        List<Song> sortedSongs = new ArrayList<>(playlist);
        Collections.sort(sortedSongs);
        System.out.println(sortedSongs);
        System.out.println(playlist);
    }
}
```

Output: [Moonlight (4.0 min), Sunrise (3.5 min)], then [Sunrise (3.5 min), Moonlight (4.0 min)]. The list copy contains the same Song references in a separate list. Sorting moves references; it does not clone songs. compareTo need not return exactly -1 or 1.
Two songs with the same title compare as zero here even if they are different objects. This does not override equals(). Natural ordering consistent with equals is recommended; equality design matters when using sorted sets/maps. Comparator can provide other orderings, such as duration, in a later lesson.

11. Assignments and practice

Complete Listenify: album creation, duplicate detection, playlist additions by title and 1-based track number, listing, next/previous, replay and removal. Handle invalid input, an empty playlist and navigation boundaries. Commit the project to GitHub.
Verify that new String("Sunrise") finds an equal title, removing a playlist entry leaves the album song intact, and removing the first/last/only song does not crash.
Java practice: predict both overloaded calls before running TypePromotionDemo; explain which references remain after each line of GarbageCollectionDemo; implement Comparable<Song> and sort a separate song list by title.
Complete the Bank exception self-practice, Vehicles practice and 15 exception MCQs from the preceding page.

12. Reading and reference resources

https://www.w3resource.com/java-exercises/oop/index.php
Type promotion and overloading — roadmap reading
Java garbage collection — roadmap reading
Comparable interface — roadmap reading
Java conversion rules
Java Comparable contract
Java garbage collection request)
Java ListIterator